# 2주차 3일차: TwoWay와 속성 변경 알림 — 30분

## 1. 학습 목표 — 2분

- 입력값을 C# Input 속성에 연결합니다.
- INotifyPropertyChanged로 결과와 상태 변경을 화면에 알립니다.

시작 상태는 day06입니다.

## 2. 핵심 개념 설명 — 7분

OneWay는 데이터 → 화면, TwoWay는 데이터 ↔ 화면입니다. 입력창은 사용자가 편집하므로 TwoWay로 연결합니다. 다만 방향과 전달 시점은 별개입니다.

TextBox.Text는 기본적으로 포커스를 잃을 때 데이터에 편집값을 반영합니다. 오늘은 UpdateSourceTrigger=PropertyChanged를 지정하여 타이핑할 때마다 Input에 반영합니다. TextBox 갱신 시점 공식 설명

반대 방향의 변경도 알릴 필요가 있습니다. C#에서 Result나 Status를 바꿀 때 INotifyPropertyChanged의 PropertyChanged 이벤트로 "이 속성이 바뀌었다"고 알립니다. 바인딩은 알림을 받고 값을 다시 읽습니다. 변경 알림 공식 설명

| 코드 | 하는 일 |
|---|---|
| Input의 set | 입력창에서 넘어온 값을 저장 |
| Result의 private set | 클래스 내부 처리 코드만 결과를 변경 |
| Notify(nameof(Result)) | Result가 바뀌었음을 알림 |

nameof(Result)는 속성 이름을 문자열로 얻습니다. event의 ?와 ?.Invoke는 등록된 대상이 없으면 호출을 건너뛰기 위한 C# 표기입니다. 이 반복 코드는 전체 파일에서 가져오고, "값 저장 다음에 알림"이라는 순서만 이해하세요.

## 3. 짧고 이해하기 쉬운 코드 예제 — 5분

```
<TextBox Text="{Binding Input, Mode=TwoWay, UpdateSourceTrigger=PropertyChanged}" />
```

Result 속성과 알림 메서드는 다음 역할을 합니다.

```
    private string result = "아직 실행하지 않았습니다.";
    public string Result
    {
        get => result;
        private set
        {
            result = value;
            Notify(nameof(Result));
        }
    }
```

```
    public event PropertyChangedEventHandler? PropertyChanged;

    private void Notify(string propertyName)
    {
        PropertyChanged?.Invoke(this, new PropertyChangedEventArgs(propertyName));
    }
```

실제 화면 배치는 전체 파일에 있습니다. Input과 Status에도 같은 방식의 알림이 들어 있습니다. 이벤트 처리기에서는 InputTextBox.Text 대신 Input을 읽고, ResultTextBlock.Text 대신 Result를 바꿉니다.

오늘 바뀌는 파일입니다. 링크에는 생략 없는 전체 코드가 있습니다.

| 파일 | 전체 코드 |
| --- | --- |
| MainWindow.xaml | day07의 MainWindow.xaml |
| MainWindow.xaml.cs | day07의 MainWindow.xaml.cs |

본문에는 핵심 부분만 보여 줍니다. 파일을 만들거나 교체할 때는 위 링크의 **전체 코드**를 사용하세요. App 파일은 바뀌지 않습니다.

## 4. Visual Studio에서 따라 하는 실습 — 13분

1. **2분:** MainWindow 두 파일을 오늘의 전체 코드로 교체합니다. Post.cs는 바뀌지 않습니다.
2. **4분:** 클래스 선언의 INotifyPropertyChanged, 세 속성, Notify 메서드를 찾습니다. 실행 메서드가 화면 컨트롤 이름을 직접 사용하지 않는 것을 확인합니다.
3. **3분:** F5로 실행하고 입력을 바꾼 뒤 실행합니다. 어제와 달리 결과 문구가 갱신되는 것을 확인합니다.
4. **4분:** 창을 닫습니다. Input의 set 안 input=value 줄에 중단점을 걸고 다시 실행합니다. 입력창에 글자 하나를 입력했을 때 멈추는 것을 봅니다. F5로 계속 실행하고 중단점을 제거합니다. 빈 입력을 실행해 안내를 확인한 뒤 종료합니다.

진도가 다르면 저장소 루트의 터미널에서 powershell -ExecutionPolicy Bypass -File .₩sample₩Use-Stage.ps1 -Stage 6로 시작 상태를 적용할 수 있습니다. 파일 복사가 어렵다면 같은 명령의 Stage를 7로 바꾸면 오늘 완료본이 적용됩니다. 실행 중인 앱을 먼저 닫고 Visual Studio에서 변경 파일을 다시 불러옵니다. 기존 교재 파일은 자동 백업됩니다. 단계 적용 안내

## 5. 이번 학습에서 반드시 이해해야 할 핵심 내용 요약 — 3분

- TwoWay는 값이 오가는 방향, UpdateSourceTrigger는 화면값을 데이터에 쓰는 시점입니다.
- INotifyPropertyChanged는 C# 속성 변경을 화면에 알립니다.
- 바인딩이 있어도 필요한 변경 알림이 없으면 화면에 새 값이 나타나지 않을 수 있습니다.
- 실무·면접에서 자주 묻는 연결은 DataContext → public 속성 → 변경 알림입니다.

완료본: day07. 다음: 2주차 4일차.